

## loading boston dataset from SciKit learn library

```
In [1]: from sklearn import datasets
import pandas as pd

In [2]: housing=datasets.fetch_california_housing()
housing
```

```
Out[2]: {'data': array([[ 8.3252      , 41.      , 6.98412698, ..., 2.55555556,
    37.88      , -122.23    ],
    [ 8.3014      , 21.      , 6.23813708, ..., 2.10984183,
    37.86      , -122.22    ],
    [ 7.2574      , 52.      , 8.28813559, ..., 2.80225989,
    37.85      , -122.24    ],
    ...,
    [ 1.7        , 17.      , 5.20554273, ..., 2.3256351 ,
    39.43      , -121.22    ],
    [ 1.8672      , 18.      , 5.32951289, ..., 2.12320917,
    39.43      , -121.32    ],
    [ 2.3886      , 16.      , 5.25471698, ..., 2.61698113,
    39.37      , -121.24    ]]),
  'target': array([4.526, 3.585, 3.521, ..., 0.923, 0.847, 0.894]),
  'frame': None,
  'target_names': ['MedHouseVal'],
  'feature_names': ['MedInc',
    'HouseAge',
    'AveRooms',
    'AveBedrms',
    'Population',
    'AveOccup',
    'Latitude',
    'Longitude'],
  'DESCR': '. _california_housing_dataset:\n\nCalifornia Housing dataset\n-----\n\n**Data Set Characteristics:**\n\nNumber of Instances: 20640\n\nNumber of At
tributes: 8 numeric, predictive attributes and the target\n\nAttribute Information:\n    - MedInc        median income in block group\n    - HouseAge    median house age in block
group\n    - AveRooms    average number of rooms per household\n    - AveBedrms    average number of bedrooms per household\n    - Population  block group population\n    - Ave
Occup    average number of household members\n    - Latitude    block group latitude\n    - Longitude    block group longitude\n\nMissing Attribute Values: None\n\nThis dataset
was obtained from the StatLib repository.\nhttps://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html\n\nThe target variable is the median house value for California district
s,\nexpressed in hundreds of thousands of dollars ($100,000).\n\nThis dataset was derived from the 1990 U.S. census, using one row per census\nblock group. A block group is the smal
lest geographical unit for which the U.S.\nCensus Bureau publishes sample data (a block group typically has a population\nnof 600 to 3,000 people).\n\nA household is a group of peopl
e residing within a home. Since the average\nnumber of rooms and bedrooms in this dataset are provided per household, these\ncolumns may take surprisingly large values for block gro
ups with few households\nand many empty houses, such as vacation resorts.\n\nIt can be downloaded/loaded using the\nfunc: 'sklearn.datasets.fetch_california_housing' function.\n\n..
topic:: References\n\n    - Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions,\n    Statistics and Probability Letters, 33 (1997) 291-297\n')
```

the various attributes of relevance of the housing object are:

1. data
2. target
3. DESCR
4. feature\_names

```
In [3]: x=housing.data
target=housing.target
y=housing.feature_names
z=housing.DESCR
```

```
In [4]: housing_df=pd.DataFrame(x)
housing_df.columns=y
print(housing_df.head(10))
print(housing_df.shape)

   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252    41.0    6.984127   1.023810     322.0    2.555556    37.88
1  8.3014    21.0    6.238137   0.971880     2401.0    2.109842    37.86
2  7.2574    52.0    8.288136   1.073446     496.0    2.802260    37.85
3  5.6431    52.0    5.817352   1.073059     558.0    2.547945    37.85
4  3.8462    52.0    6.281853   1.081081     565.0    2.181467    37.85
5  4.0368    52.0    4.761658   1.103627     413.0    2.139896    37.85
6  3.6591    52.0    4.931907   0.951362     1094.0    2.128405    37.84
7  3.1200    52.0    4.797527   1.061824     1157.0    1.788253    37.84
8  2.0804    42.0    4.294118   1.117647     1206.0    2.026891    37.84
9  3.6912    52.0    4.970588   0.990196     1551.0    2.172269    37.84

   Longitude
0   -122.23
1   -122.22
2   -122.24
3   -122.25
4   -122.25
5   -122.25
6   -122.25
7   -122.25
8   -122.26
9   -122.25
(20640, 8)
```

```
In [5]: print(z)

.. _california_housing_dataset:

California Housing dataset
-----

**Data Set Characteristics:**

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and the target

:Attribute Information:
  - MedInc        median income in block group
  - HouseAge      median house age in block group
  - AveRooms      average number of rooms per household
  - AveBedrms     average number of bedrooms per household
  - Population    block group population
  - AveOccup      average number of household members
  - Latitude      block group latitude
  - Longitude     block group longitude

:Missing Attribute Values: None

This dataset was obtained from the StatLib repository.
https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html

The target variable is the median house value for California districts,
expressed in hundreds of thousands of dollars ($100,000).

This dataset was derived from the 1990 U.S. census, using one row per census
block group. A block group is the smallest geographical unit for which the U.S.
Census Bureau publishes sample data (a block group typically has a population
of 600 to 3,000 people).

A household is a group of people residing within a home. Since the average
number of rooms and bedrooms in this dataset are provided per household, these
columns may take surprisingly large values for block groups with few households
and many empty houses, such as vacation resorts.

It can be downloaded/loaded using the
:func:'sklearn.datasets.fetch_california_housing' function.

.. topic:: References

  - Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions,
    Statistics and Probability Letters, 33 (1997) 291-297
```

```
In [6]: # The train_test_split function helps in splitting the given dataset for training our ML algorithm
from sklearn import model_selection
print(model_selection.train_test_split(x,target))
x_train,x_test, target_train, target_test=model_selection.train_test_split(x,target)

[[array([[ 3.5156      , 31.      , 4.94584838, ..., 3.83393502,
    34.07      , -118.07    ],
    [ 4.5909      , 35.      , 5.54594595, ..., 4.07027027,
    34.23      , -118.41    ],
    [ 2.564       , 32.      , 4.18655462, ..., 4.47058824,
    34.06      , -117.99    ],
    ...,
    [ 3.4676      , 21.      , 4.32798165, ..., 2.43730887,
    34.18      , -118.87    ],
    [ 2.2542      , 43.      , 4.55462185, ..., 3.59663866,
    37.79      , -122.24    ],
    [ 3.7813      , 36.      , 6.63461538, ..., 3.03846154,
    33.84      , -118.1     ]]), array([[ 4.8269      , 25.      , 5.35135135, ..., 2.82162162,
    37.      , -121.97    ],
    [ 3.5781      , 44.      , 5.26296296, ..., 2.25555556,
    37.71      , -122.13    ],
    [ 8.5425      , 35.      , 7.50840336, ..., 2.78361345,
    37.24      , -121.98    ],
    ...,
    [ 3.7578      , 16.      , 2.53947368, ..., 4.10526316,
    34.2       , -117.38    ],
    [ 3.0139      , 39.      , 5.17573222, ..., 2.66945607,
    36.63      , -119.68    ],
    [ 3.3182      , 19.      , 6.28323699, ..., 3.75144509,
    32.86      , -115.4     ]]), array([1.993, 1.792, 1.458, ..., 2.128, 1.122, 2.091]), array([2.729 , 1.8 , 5.00001, ..., 1.375 , 0.653 , 1.138 ]])]
```

```
In [7]: from sklearn.linear_model import LinearRegression
```

```
In [8]: # creating algorithm object
alg1=LinearRegression()
alg1.fit(x_train, target_train)
target_predicted=alg1.predict(x_test)
print(target_predicted)

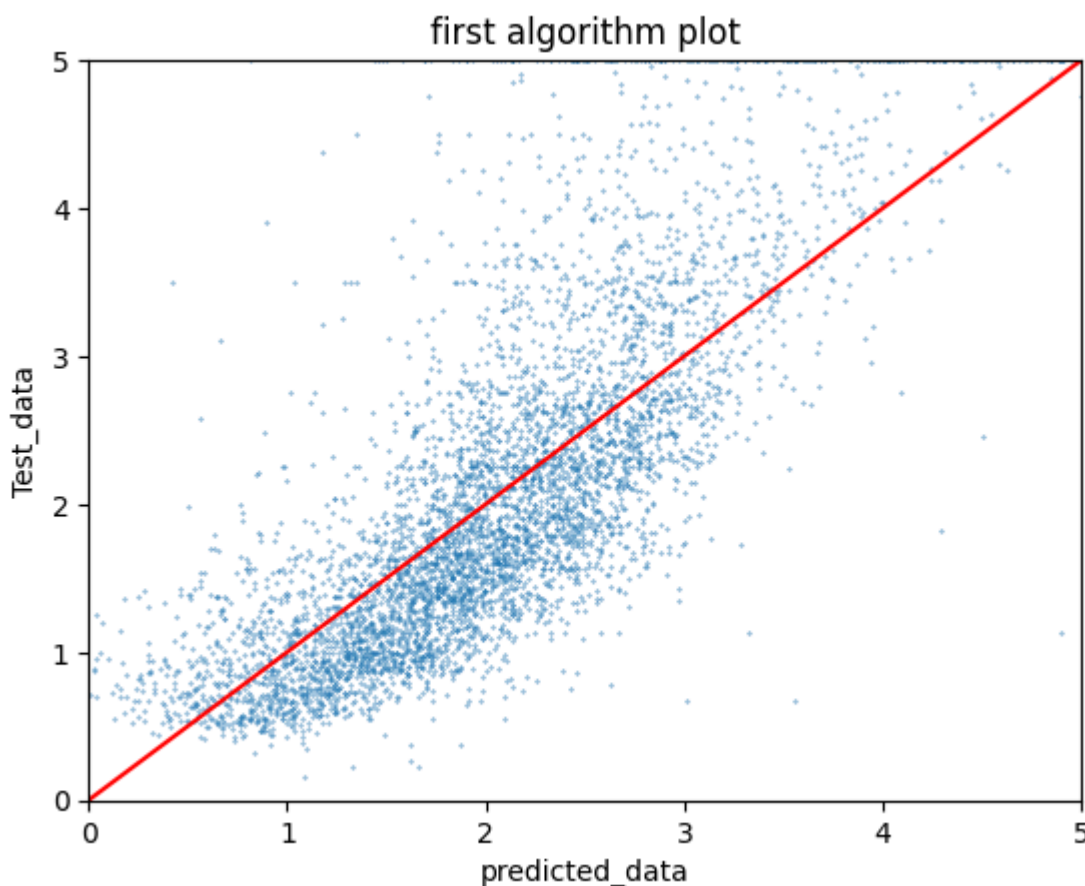
"""
the parameters of the line predicted by the linear regression algorithm
"""
print(alg1.coef_)#the coefficients of the input parameter terms
print(alg1.intercept_)#the intercept of the linear function predicted by LIN reg algo

[ 2.46947711  0.94498032  3.39559165 ... -0.09620134  2.49059413
 1.27301557]
[ 4.38309847e-01  9.61767340e-03 -1.06859512e-01  6.39735325e-01
 -2.01709297e-06 -4.69030863e-03 -4.17995254e-01 -4.30938375e-01]
-36.64290501600356
```

```
In [9]: import matplotlib.pyplot as plt

plt.plot([0,1,5],[0,1,5], 'r-')#y=x line to comare predicted and actual outputs
plt.axis([0,5,0,5])
plt.scatter(target_predicted, target_test, s=1)
plt.title('first algorithm plot')
plt.ylabel('Test_data')
plt.xlabel('predicted_data')
plt.show()

"""
the plot shows that the linear regression algo is working well enough as most of the data points lie near the line y=x
although there are a few stray points, this can get better with more sophisticated algorithms.
"""
```



```
Out[9]: '\nthe plot shows that the linear regression algo is working well enough as most of the data points lie near the line y=x\nalthough there are a few stray points, this can get better
with more sophisticated algorithms.\n'
```

```
In [10]: alg1.score(x_train, target_train)
#calculates the coeff of determination for based on the target values it predicts and the given correct target vales
```

```
Out[10]: 0.6051960930282907
```



